Simulation-Based Estimation of General Structural Network Models

Cameron Fen*

February 19, 2023

This paper addresses the issue of estimating structural models with data from a single graph, which is a longstanding problem in economics and related fields. While there exist methods for estimating structural models on independent and identically distributed graphs, there is limited research on a general-purpose algorithm to fit a structural model on a single network. This study proposes an algorithm adapted from deep learning for network analysis, which enables Bayesian and likelihood estimation of arbitrary structural models, provided that the graph data is within the range of the networks simulated by the structural model. Simulated data tests demonstrate the effectiveness and accuracy of this approach. Additionally, the algorithm is applied to estimate a homophily citation model (Bramoullé et al., 2012), making it the first method, as far as the author is aware, capable of structurally estimating this model. Overall, this study presents a promising algorithm for fitting structural models on a single network, opening up avenues for future research in the field.
JEL Codes: C11, C68, C63, C45
Keywords: Neural Networks, Bayesian Inference, Network Estimation, Structural Models, Simulation-Based Estimators

*Cameron Fen is PhD student at the University of Michigan, Ann Arbor, MI, 48104 (E-mail: camfen@umich.edu. Website: cameronfen.github.io.). The author thanks Florian Gunsilius for helpful feedback. All errors are my own.

I. Introduction

The estimation of structural models on networks is a less developed field compared to dynamic models. Current methods are specialized and cannot be applied to arbitrary structural models. The need for a general-purpose structural network estimation procedure is necessary, as the Method of Simulated Moments (MSM) (McFadden, 1989) did for macroeconomics and other fields. However, applying MSM on networks requires data on multiple independent and identically distributed (iid) networks generated from the same model (Banerjee et al., 2013). Since nodes and edges in networks are not iid, aggregation of moments over nodes or edges to perform an MSM-like approach is impossible.

This paper proposes an algorithm that samples from the parameter priors and the network model conditional on parameter samples. The network model is then converted into sufficient statistics through a graph neural network or a graph embedding, allowing the use of density estimators. The algorithm can estimate arbitrary structural models on a single real-world graph, and MLE and Bayesian estimates for single or multiple graphs. Current methods rely on either tailored models or reduced-form techniques that yield a likelihood function based on sufficient statistics, making estimation on real data difficult.

The approach advocated in this paper follows the approach in Fen (2022). One first samples from the parameter priors. Then one sample from the network model is conditional on the parameter samples $X' \sim P(X'|\theta)$. The Fen (2022) approach then uses a machine learning density estimator to learn $P(\theta|X')$ from the joint samples $X', \theta \sim P(X', \theta) = P(X'|\theta)P(\theta)$. However, in order to use a density estimator on X', one needs to convert the graph sampled from a structural model into sufficient statistics as the density estimators used in Fen (2022) work only on numbers. The way I go about this is to take either a graph neural network or a graph embedding in order to convert the graph into a set of sufficient statistics. After running the density estimator to recover $P(\theta|X')$, I also run the true graph, X, through the embedding and condition the density estimator on the true embedding, $P(\theta|X' = X)$, which returns the posterior. If one wants to perform MLE estimation one can use flat priors and take the mode of the posterior.

There are many papers that have implemented structural models, but due to lack of estimation tools, do not perform full estimation on real data (Calvo-Armengol and Jackson, 2004), (Carvalho and Voigtländer, 2014), (Calvó-Armengol and Jackson, 2007). Because conventional estimation techniques are generally ad-hoc or only work for data with multiple networks, many papers choose not to estimate their models on data and only report theoretical properties of their models (Jackson and Wolinsky, 1996), (Gilles, Johnson et al., 2000), (Furusawa and Konishi, 2007).

The proposed algorithm is efficient, works on a single network, and performs MLE without constructing a likelihood function. It works on simulated data and can estimate the homophily citation model, a model that hasn't been estimated before. Unlike other approaches, the algorithm does not require a provided likelihood, is faster than brute force grid search, especially for large models, and doesn't require an ad-hoc choice of reduced form network statistics like the Exponential Random Graph Model (Robins et al., 2007). However, the maximum likelihood solution might not be unique if the embedding network does not return a unique embedding for each possible graph.

II. Methods

In this section, we will present an overview of the simulation-based estimation approach, the Sequential Neural Posterior Estimation (SNPE) algorithm. The SNPE algorithm is a Bayesian inference technique that uses a machine learning conditional density estimator to learn the posterior distribution of the parameters of interest. In addition to the SNPE algorithm, I will also discuss embedding networks. Embedding networks are used to convert graph-structured data into a set of sufficient statistics that can be used as inputs to the density estimator in the SNPE algorithm. Two common types of embedding networks are graph convolutional neural networks (Kipf and Welling, 2016) and FEATHER (Rozemberczki and Sarkar, 2020). Graph convolutional neural networks use convolutional layers to process the graph structure and generate sufficient statistics. On the other hand, FEATHER is an algorithm that uses the characteristic function of a random walk to generate sufficient statistics for graphs. For more information on normalizing flow models and the SNPE algorithm, as well as Metropolis-Hastings, the reader is referred to the section "Background on

Simulation Neural Posterior Estimation and Metropolis-Hastings" in Fen (2022).

II.A. Background on Simulation Neural Posterior Estimation (SNPE)

The application of on SNPE for simulation-based Bayesian inference has gained increasing attention in the field of economics. The SNPE algorithm has been described in detail in Fen (2022), which provides a comprehensive account of the method. This paper will only provide a brief overview of the algorithm, as well as a discussion of the graph neural network and other embedding methods that are used to convert graph data into sufficient statistics.

SNPE is a simulation-based inference method that can recover the posterior distribution by sampling from the parameters space, $P(\theta)$, and the model, $P(X'|\theta)$, and estimating the density $P(\theta|X')$. The estimation of the density is done by fitting a density estimator of the conditional posterior on the joint samples (X', θ). The density estimators will be briefly discussed below in this section but more comprehensively in Fen (2022). The resulting posterior estimate is conditioned on the real data, X, and can be used to make predictions or inferences about the parameters of the model. The SNPE algorithm shares similarities with the approach to simulation-based MLE proposed by Kristensen and Shin (2012).

To apply SNPE in the case of graph data, it is necessary to use a graph neural network or a graph embedding algorithm to convert the network data into a set of sufficient statistics. This is because in this case, X' and X are model-simulated network data and true data, respectively, and are in graph form. The graph neural network converts the graph data into sufficient statistics for the normalizing flow to train on, whereas the FEATHER algorithm (Rozemberczki and Sarkar, 2020) uses the characteristic function of a random walk to create sufficient statistics.

Algorithm 1: SNPE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, flow $f_\phi(x|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(x, \theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Train $f_\phi(\theta|x)\dfrac{p^{(i-1)}}{p(\theta)}$ on $\{x^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)} \propto f_\phi(\theta|x)$;

end

The methodology employed in this study builds on previous research that utilized the kernel density estimator (KDE) for simulation-based inference (Kristensen and Shin, 2012). While the KDE has been shown to be a flexible and versatile method, it is not without limitations. In particular, the KDE may struggle with high-dimensional conditional densities, leading to poor estimation accuracy and slow computation times.

To address these limitations, this study utilizes advanced machine-learning techniques to estimate high-dimensional densities and conditional densities. The methods discussed in Fen (2022) offer several promising alternatives to the traditional KDE approach. One such method is the normalizing flow, which is able to handle high-dimensional data and estimate conditional densities. Another approach discussed in the paper is the use of a GAN, which can estimate the likelihood function, but requires either rejection sampling or variational inference projection onto a flow for sampling. A third approach involves using a conditional mixture of Gaussians, which is more straightforward but may not be the most effective approach. Since it is the easiest approach to follow and works well, I will focus on this approach.

The conditional mixture of Gaussians approach is a powerful tool for density estimation in high-dimensional spaces, and in the context of simulation-based Bayesian inference, can enable the estimation of conditional posteriors. As noted in Fen (2022), this approach has several advantages over other density estimators such as normalizing flows and GANs. In particular, it is easier to implement and requires fewer samples to obtain accurate density estimates. As outlined in Bishop (1994), a mixture of Gaussians is a linear combination of N Gaussian distributions, each with its own mean, covariance, and weight. The probability density function (pdf) of a point under this mixture can be calculated using the equation:

$$P(\theta) = \sum_i \pi_i * Q_i(\theta)$$

To obtain the conditional density estimator, a neural network is used to model the conditional relationship between the parameters θ and the data X'. The neural network takes X' as input and returns the parameters for the mixture of Gaussians, including the weights, means, and covariances for each Gaussian component. These parameters can then be used to estimate the conditional posterior density $P(\theta|X')$ using the equation provided in the previous paragraph:

$$P(\theta) = \sum_i \pi_i * Q_i(\theta)$$

The use of a neural network to estimate conditional density also provides flexibility and adaptability to the density estimator. Small changes in the input data X' can result in small changes to the mixture of Gaussians, allowing for the model to adapt to changes in the data. Additionally, the parameters of the neural network can be optimized via MLE to improve the accuracy of the density estimator. This approach allows for the estimation of accurate conditional posteriors, even in high-dimensional spaces, and can be a useful tool for simulation-based inference.

II.B. Graph Convolution Neural Networks

In order to use graphs as inputs for density estimation, they must first be converted into numerical values or sufficient statistics. This is typically achieved using graph neural networks or graph

embeddings. One type of neural network is the feed-forward neural network, which is composed of multiple layers of interconnected nodes. Each layer takes the output of the previous layer and maps it to a new set of outputs. The basic unit of a feed-forward neural network is the layer, which is defined by the equation:

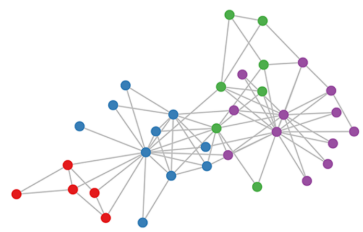
$$x^j = \sigma(A_i x^i + B_i) \tag{1}$$

Here, x^i and x^j represent the input and output of the layer, respectively. The matrix A_i maps the input to the output and B_i is a bias term. The function σ is a nonlinearity that adds flexibility to the network, allowing it to learn complex relationships between inputs and outputs. A popular choice of nonlinearity is the rectified linear unit (ReLU), which has been shown to work well in many applications (Agarap, 2018).

While a feed-forward neural network can be used to process a variety of inputs, including graphs, it does not take into account the inherent structure of the input. Graphs have a natural structure that can be exploited to improve the accuracy of the density estimator. This is where graph convolutional neural networks (GCNs) come in. GCNs are a type of neural network designed specifically to work with graph-structured data. They use a form of convolution operation, similar to that used in image processing, to extract features from the graph. This allows the network to take into account the topology of the graph, as well as the features of individual nodes and edges when making predictions (Wu et al., 2020).

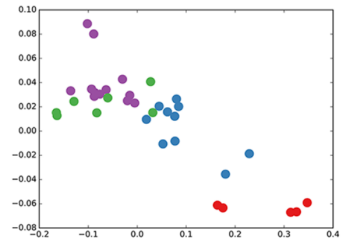
Graph convolutional neural networks (GCNN) (Kipf and Welling, 2016) have shown great potential in working with graph data, and their architecture is specifically designed for this purpose. Typically, GCNNs define an adjacency matrix, denoted by A, which characterizes the edges between nodes in a graph. This matrix is usually normalized by dividing by the degree matrix, and the resulting normalized matrix is denoted by A^*. In GCNNs, the node characteristic matrix X^0 is first defined, and then the node embedding characteristic matrix X^j is obtained via the equation $X^j = \sigma(W_i X^i A^* + B_i)$, where W_i and B_i are learnable parameter matrices (vectors), and σ is

a nonlinearity. By using the adjacency matrix, GCNNs extend the feed-forward neural network architecture to enable the characteristics of nearby nodes to affect each other more strongly than those of far away nodes. This inductive bias allows the model to efficiently handle data that is defined on networks. For instance, a randomly initialized graph neural network can nearly learn how to cluster network data properly, as demonstrated in the karate club dataset, which is classified by color using a modularity-based algorithm (Brandes et al., 2007). These colors represent clusters of nodes that share a community together, due to sharing an abnormally high number of edges within the cluster. The below shows the karate club data set with nodes colored according to their cluster:



Karate club graph, colors denote communities obtained via modularity-based clustering (Brandes et al., 2008).

The randomly initialized GCNN then produces a 2D embedding and even without training, nodes belonging to the same cluster are given embedding values close to one another. Below shows the same cluster colors produced by a randomly initialized graph neural network:



GCN embedding (with random weights) for nodes in the karate club network.

With training, one can learn even more effective embeddings that can take a graph and produce sufficient statistics for the simulation-based inference approach. Given the normalizing flow trained to estimate $P(\theta|X')$ and a graph G, the GCNN takes G and returns a numerical embedding X'

that can be fed into the density estimator to properly condition on the graph.

II.C. FEATHER Graph Embedding

In the field of graph representation learning, one common approach is to use a graph neural network to learn a good embedding for each node and the overall graph. However, an alternative is to use an out-of-the-box graph embedding technique without learning parameters. In this paper, I use the FEATHER algorithm (Rozemberczki and Sarkar, 2020) as the graph embedding technique. FEATHER learns node embeddings from the characteristic function of the probability density function (pdf) implied by a random walk from a node, and then averages the node embeddings to obtain the graph embedding. This is achieved by learning the function that maps the node-level features, which in our case are the default features of eccentricity, transitivity, and degree, to the complex plane. The algorithm then evaluates the random walk on the graph for a certain number of iterations and at designated points of θ in order to obtain the characteristic function. The graph-level embedding is obtained by averaging the node embeddings.

For each node, the FEATHER algorithm attempts to learn the characteristic function for the probability distribution of a random walk starting from the source node. Given a graph G with vertex and edges V, E, the FEATHER algorithm learns the function:

$$E[e^{i\theta x}|G, u] = \sum_{w \in V} P(w|u)(\cos(\theta x_w) + i \sin(\theta x_w))$$

Here $P(w|u)$ is the probability that node w is reached from node u after r random walk steps. x_w are the node-level features. As my graphs have no node features, the features are the default features of eccentricity, transitivity, and degree. One can evaluate the random walk on a graph for r iterations from 1 to n steps from the original node. Then one can also evaluate the characteristic function at designated points of θ. In order to get a graph-level embedding from these node embeddings, the algorithm averages across node embeddings.

While FEATHER is the embedding technique used in this paper, there are many other approaches that can be used. Other graph embedding techniques include graph2vec (Narayanan

et al., 2017), wavelets-based embeddings (Wang et al., 2021), and many others from libraries such as the Karate Club (Rozemberczki, Kiss and Sarkar, 2020). The reason for using a fixed embedding is that, unless one is dealing with sparse graphs, the amount of memory required to store a graph is quadratic in the number of nodes. As our method, Sequential Neural Posterior Estimation (SNPE), often requires a large number of graphs for estimation, memory constraints can quickly become an issue. However, with an embedding method that does not require any modifiable weights, we can derive the embedding and then throw away the graph, saving only the embedding for future iterations. This allows us to work with much larger graphs without running into memory constraints.

II.D. Limitations of Graph-based Embedding Methods

This section discusses the main weakness of GCNNs, and related techniques like graph transformers, and graph embedding methods. One main result discussed in Fen (2022) concerns universal approximator properties of flows. The graph SNPE algorithm does not have this quality. Likewise, large social networks are also beyond the computational capacity of these GCNNs, although with fixed embeddings work on them is likely possible.

The results in Fen (2022) and borrowed from Huang et al. (2018), Goodfellow et al. (2020), and Bishop (1994) demonstrate that the normalizing flow, GAN, and a mixture of Gaussians in the SNPE algorithm are universal approximator of probability distributions and that any large enough density estimator of these forms can model any continuous probability distribution. However, this result no longer holds with graph data. The reason is that GCNNs, their cousins graph attention transformers (Veličković et al., 2017), and related embedding methods, can not distinguish between certain graphs that are non-isomorphic (Xu et al., 2018). Thus they are not universal approximators.

Although the result of universal approximation no longer holds, graph convolutional neural networks are a powerful model that has useful inductive biases for embedding graphs. While the results in Papamakarios and Murray (2016) suggest that SNPE converges to the Bayesian posterior, the assumption that the conditioning variable captures all variability in the data is not met when using most graph-based embedding models–be they GCNNs or embedding algorithms. Nevertheless,

these approaches are specifically tailored toward graphs and give much better performance on graphs than a feed-forward network.

III. Results

This section presents empirical results that demonstrate the efficacy of the proposed algorithm in recovering the parameters of a structural model from the graph generated by those parameters. To this end, I begin by discussing the Newman-Watts-Strogatz graph (Newman and Watts, 1999), a widely-used network model. The algorithm is then tested on other network models, namely the Power-law Cluster (Holme and Kim, 2002) and Relaxed Caveman graphs (Fortunato, 2010). Finally, I perform an empirical excercise estimating the Bramoullé et al. (2012) structural model on the Karate Club network.

To evaluate the algorithm's performance, simulated graphs were drawn from the structural model using a specific set of parameters, and the structural model parameters were then estimated based on the given graph. The results are presented in the charts below, where the blue line corresponds to the posterior of the estimation routine, and the red line represents the true parameter value. The off-diagonal charts show a heat map of the two-way distribution implied by the parameter that defines the row and the column that intersects on a given chart. In cases where the parameters are integer-valued, a continuous uniform distribution was still used for sampling, but the floor function was applied to generate an integer. As the number of nodes is known both in the dataset and in the model, the algorithm generates a graph with the same number of nodes as the dataset.

The results demonstrate the algorithm's effectiveness in recovering the structural model parameters from the generated graph for all four types of network models considered. The posterior estimates closely match the true parameter values, as indicated by the convergence of the blue lines to the red lines in the charts. The heat maps further confirm the accuracy of the parameter estimation, with high probability density concentrated around the true parameter values. These findings provide evidence that the proposed algorithm can be applied to a wide range of network models and can effectively recover the underlying structural parameters, which is crucial for accurately

characterizing and understanding real-world networks.

III.A. Newman-Watts-Strogatz Small World Graph

The focus of this section is to discuss the results obtained for the small-world graph, as proposed by Newman and Watts (1999). This model has two parameters, namely, k and p. In this model, the number of nodes is arranged in a ring, and each node is connected to the k nearest neighbors in the ring. For each edge, there is a probability p that the terminating node u of that edge u, v will form a new edge with a random node w. To embed the generated graph, a graph convolutional neural network is employed. In our analysis, the value of k is fixed at 12, and p is set to 0.33. The results indicate that the proposed algorithm successfully recovers a posterior distribution whose mode is quite close to the true parameter values of k and p, demonstrating the algorithm's efficacy in recovering the underlying structural parameters for this specific type of graph.

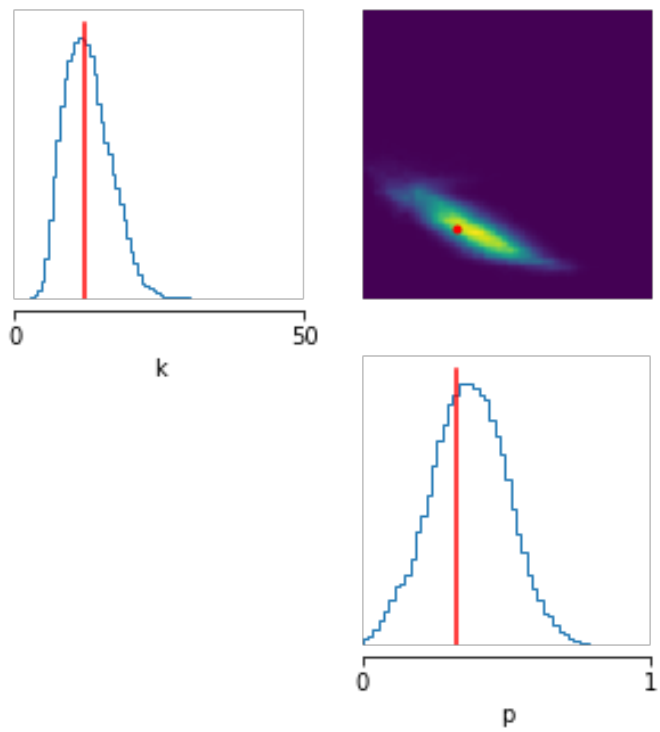


Figure I: Newman-Watts-Strogatz Small World Graph

III.B. Power Law Cluster Graph

In this section, I will focus on the estimation of the power law cluster graph on simulated data, as proposed by Holme and Kim (2002). The model has two parameters: the number of random edges to add to each node and the probability of closing a triangle after an edge is added. The random edge parameter must be an integer; therefore, uniform random variables are generated and then rounded to the nearest integer. For our analysis, the random edge parameter is set to 3, and the triangle probability is set to 0.35. These parameter values result in fewer triangles being created, as the parameter values are on the low side. The results indicate good identification of the edge parameter, but the triangle parameter is less well identified. This is consistent with the literature on Exponential Random Graph Models (ERGM), where it is widely known that triangles are difficult to identify (Handcock et al., 2020). To embed the generated graph, a graph convolutional neural network is employed. Overall, the results suggest that the proposed algorithm can successfully estimate the edge parameter in the power law cluster graph, despite challenges in identifying the triangle parameter.

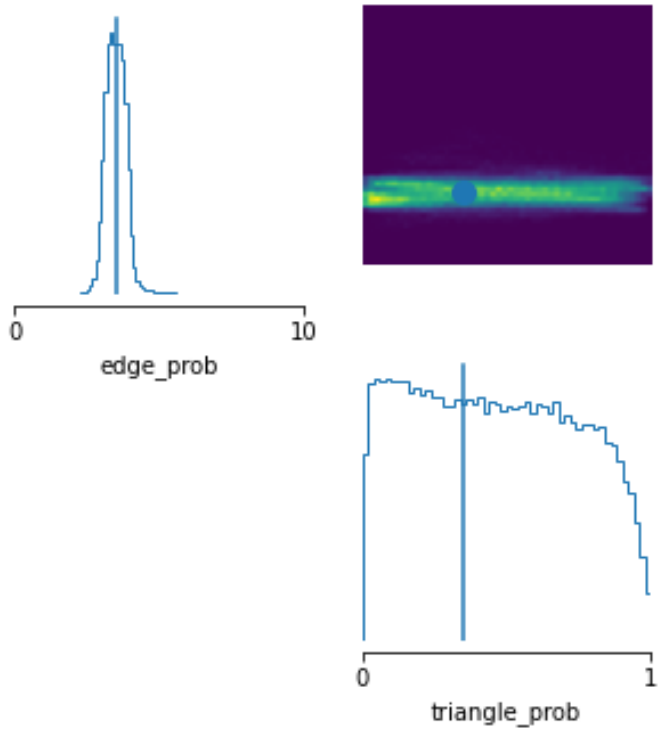


Figure II: Power Law Cluster Graph

This chart shows the model accurately recovering the random edge probability of 3, However, there is difficulty recovering the triangle probability as mentioned above.

III.C. Relaxed Caveman Graph

In this section, we turn our attention to the Relaxed Caveman Graph proposed by Fortunato (2010), which is formed by randomly rewiring an edge formed by a clique in a caveman graph with probability p. To generate the Relaxed Caveman graph, n cliques of size m are formed, and then one edge in each clique is deleted to form another edge that connects all the cliques in a cycle. For the analysis, I use 10 cliques of size 10 with a rewiring probability of 60 percent. The high rewiring probability is intended to make the graph look less distinctive and resembles graphs seen in nature more often. In contrast to the graph convolutional neural network used in previous sections, I

use the FEATHER graph embedding proposed by Rozemberczki and Sarkar (2020) to embed the relaxed caveman graph.

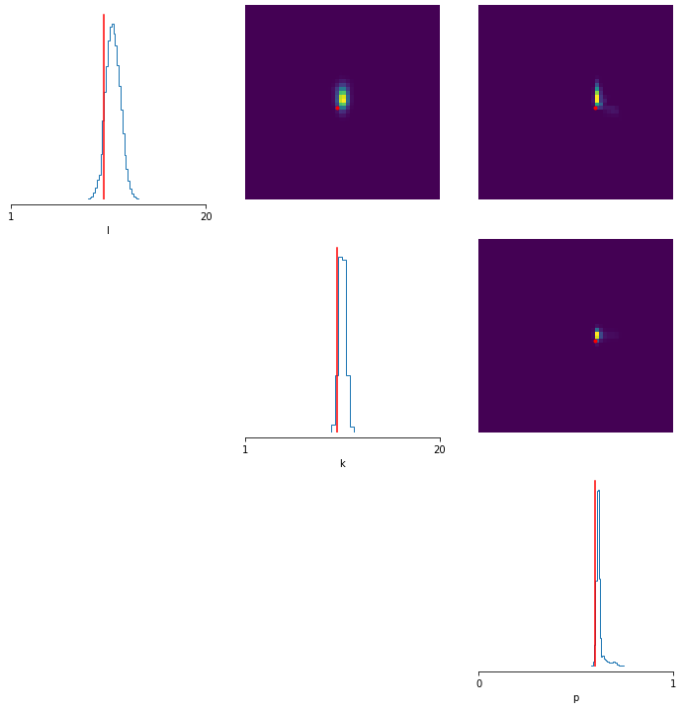


Figure III: Relaxed Caveman Graph

The posterior distribution for the relaxed caveman graph shows good convergence of the algorithm, with the true parameters being very close to the mode of the posterior distribution. However, there is a slight shift in the mode of the posterior distribution from the true value. This is a mistake that can often be made using SNPE. Although Metropolis-Hastings Markov Chain Monte Carlo (MCMC) often has poor mixing problems, SNPE algorithms rely on sampling and density estimation and errors usually involve the entire distribution being shifted slightly from the correct location. Overall, the results suggest that the proposed algorithm can successfully estimate the parameters of the relaxed caveman graph, with some minor deviations in the estimated posterior distribution.

The above figure shows the relaxed caveman graph generated by my algorithm, which is in line

with the expected structure of the relaxed caveman graph. The graph comprises ten cliques, with each clique having ten nodes, and the edges between cliques are randomly rewired. The generated graph demonstrates the ability of the algorithm to recover the underlying structure of the graph and estimate its parameters.

IV. Empirical Application: Citation Graphs

This section of the economic research article outlines an empirical application of an algorithm to real data based on the citation graph homophily paper by Bramoullé et al. (2012). The algorithm is used to extend the results of the original paper by structurally estimating the model, which was not done in the original study.

The paper by Bramoullé et al. (2012) did not estimate their model on real data. Instead, they use citations and citations of cited papers to come up with statistics that approximate what the true underlying parameter value is. The paper assumes that n nodes are born one after another. Each node that is born sends $m > 1$ connections to previously born nodes. A fraction of these links m_r are connections at random, and a fraction of these links $m_s = m - m_r$ are search links–connections formed by citing a paper cited by a paper you cited.

However, this approach is less realistic for a graph of friends in the karate club data set since the number of friends one has does not have a temporal dimension. Therefore, I modify the Bramoullé et al. (2012) model to be more applicable to this scenario. In the modified model, all nodes are initialized at the beginning and there is a probability that each node has a direct connection with another node at the beginning of each round. Additionally, there is a separate probability that a node is connected to the connection of a connection. Since temporal relationships have been destroyed, this is an undirected network with the node representing a citation from the earlier paper to the later paper. There are 15 rounds in total which is a reasonable amount for the 34 node karate club dataset.

The original paper by Bramoullé et al. (2012) did not provide an estimation procedure for their approach, so they used statistics from the graph to approximate the random connection and search

connection rate. They estimated the search homophily probability as any citation that is cited but also has an intermediary that is also cited by and cites the search citation. The remaining citations are classified as random citations. However, if the data was generated by this model, this approach could overcount search citations if there are some citations that are randomly created even though there is an intermediary that is cited by the citing paper and cites the cited paper. On the other hand, if the model is not entirely correct, it could also undercount search citations if someone cites the paper knowing an intermediary paper but does not cite the intermediary paper.

IV.A. Karate Club Friendships Graph

This paper presents an analysis of a homophily model applied to the karate club dataset, a dataset of friendship connections among karate club participants. The Bramoullé et al. (2012) model assumes that individuals make direct friends and then make friends from their friends' friends. To test the validity of the model, I estimate it on the empirical data and then generate simulated data using the model's estimated parameters. I then estimate the model on the simulated data to check for convergence to the true solution.

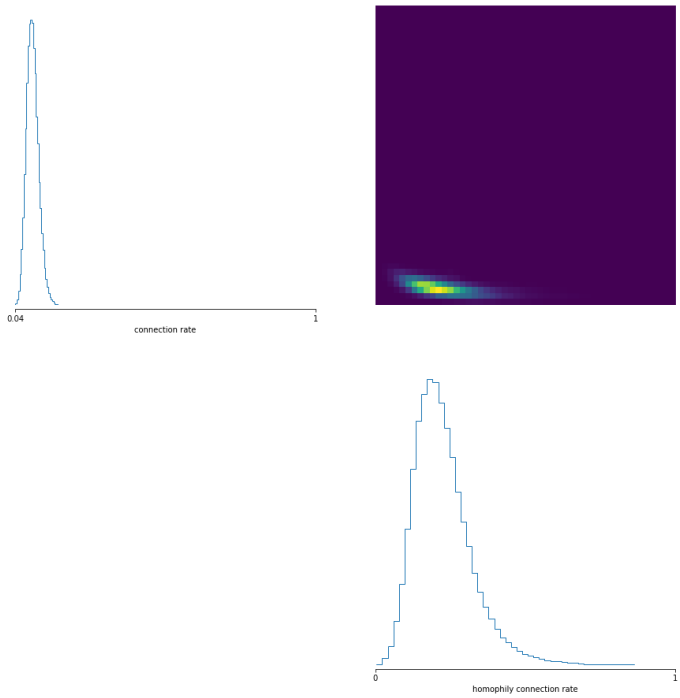


Figure IV: Homophily Model estimated on Karate Club data

I estimate the model on the karate club dataset and obtain the posterior distribution shown in the above figure. This figure displays the homophily model estimated on the karate club data. The authors argue that this modification allows for identification, as the empirical data can be generated by the structural model. I then estimate the model on the simulated data. To recover the true parameters on the simulated data, I set the random connection rate to .08 and the homophily search connection rate to .23, which were the approximate means of the empirical posterior. The results of this estimation are shown in the figure below:

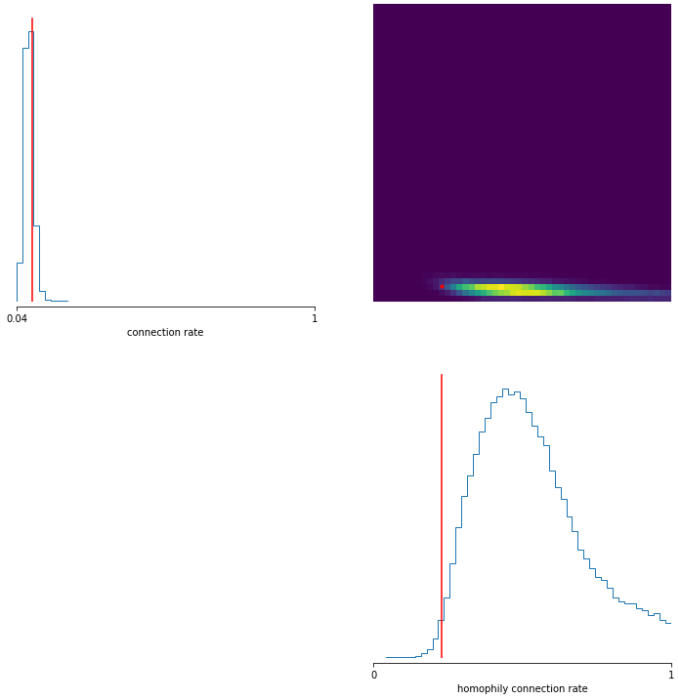


Figure V: Homophily Model estimated on Homophily simulated data

From the pictures, it is evident that the model does a good job of identifying the random connection parameters. However, the model performs poorly in identifying the homophily search parameter. I attribute this to the fact that the simulated data, in line with the karate club data, has only 34 nodes, and there will be some variance in the simulated data, and a connection of a connection will be more difficult to extract from data than direct connections. Nevertheless, the true parameter is still within the support of the posterior for the homophily search parameter. These findings suggest that the homophily model provides a reasonable explanation of the observed friendship connections in the karate club dataset, as well as provides insights into the formation of social networks more generally.

V. Conclusion

In conclusion, this paper has introduced a novel method to estimate structural network models using a single network. The approach is general-purpose and can estimate many models, regardless of the structure, as long as one can simulate graphs from the structural model. Although this approach is relatively unexplored, the results of this study demonstrate that it is a promising approach that can lead to a better structural understanding of networks in the real world. However, this approach is not without its limitations. One of the central issues is to clarify both theoretically and empirically what sort of network models this approach can effectively estimate and which approaches it fails. Additionally, many models have to be restructured so that the model puts positive support on the data used for estimation. Nevertheless, there are many models that have not been estimated in the literature that can be estimated in this way with some modifications to adjust for the support issue. In light of these potential avenues for further research, this approach seems like a promising tool for future analysis and has the potential to advance the study of network models.

References

Agarap, Abien Fred. 2018. "Deep learning using rectified linear units (relu)." *arXiv preprint arXiv:1803.08375*.

Banerjee, Abhijit, Arun G Chandrasekhar, Esther Duflo, and Matthew O Jackson. 2013. "The diffusion of microfinance." *Science*, 341(6144): 1236498.

Bishop, Christopher M. 1994. "Mixture density networks."

Bramoullé, Yann, Sergio Currarini, Matthew O Jackson, Paolo Pin, and Brian W Rogers. 2012. "Homophily and long-run integration in social networks." *Journal of Economic Theory*, 147(5): 1754–1786.

Brandes, Ulrik, Daniel Delling, Marco Gaertler, Robert Gorke, Martin Hoefer, Zoran Nikoloski, and Dorothea Wagner. 2007. "On modularity clustering." *IEEE transactions on knowledge and data engineering*, 20(2): 172–188.

Calvo-Armengol, Antoni, and Matthew O Jackson. 2004. "The effects of social networks on employment and inequality." *American economic review*, 94(3): 426–454.

Calvó-Armengol, Antoni, and Matthew O Jackson. 2007. "Networks in labor markets: Wage and employment dynamics and inequality." *Journal of economic theory*, 132(1): 27–46.

Carvalho, Vasco M, and Nico Voigtländer. 2014. "Input diffusion and the evolution of production networks." National Bureau of Economic Research.

Fen, Cameron. 2022. "Fast Simulation-Based Bayesian Estimation of Dynamic Models using Normalizing Flow Neural Networks."

Fortunato, Santo. 2010. "Community detection in graphs." *Physics reports*, 486(3-5): 75–174.

Furusawa, Taiji, and Hideo Konishi. 2007. "Free trade networks." *Journal of International Economics*, 72(2): 310–335.

Gilles, Robert P, Cathleen Johnson, et al. 2000. "original papers: Spatial social networks." *Review of Economic Design*, 5(3): 273–299.

Goodfellow, Ian, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2020. "Generative adversarial networks." *Communications of the ACM*, 63(11): 139–144.

Handcock, Mark S, David R Hunter, Carter T Butts, Steven M Goodreau, Pavel N Krivitsky, and Martina Morris. 2020. "ergm: Fit, Simulate and Diagnose Exponential-Family Models for Networks."

Holme, Petter, and Beom Jun Kim. 2002. "Growing scale-free networks with tunable clustering." *Physical review E*, 65(2): 026107.

Huang, Chin-Wei, David Krueger, Alexandre Lacoste, and Aaron Courville. 2018. "Neural autoregressive flows." 2078–2087, PMLR.

Jackson, Matthew O, and Asher Wolinsky. 1996. "A strategic model of social and economic networks." *Journal of economic theory*, 71(1): 44–74.

Kipf, Thomas N, and Max Welling. 2016. "Semi-supervised classification with graph convolutional networks." *arXiv preprint arXiv:1609.02907*.

Kristensen, Dennis, and Yongseok Shin. 2012. "Estimation of dynamic models with nonparametric simulated maximum likelihood." *Journal of Econometrics*, 167(1): 76–94.

McFadden, Daniel. 1989. "A method of simulated moments for estimation of discrete response models without numerical integration." *Econometrica: Journal of the Econometric Society*, 995–1026.

Narayanan, Annamalai, Mahinthan Chandramohan, Rajasekar Venkatesan, Lihui Chen, Yang Liu, and Shantanu Jaiswal. 2017. "graph2vec: Learning distributed representations of graphs." *arXiv preprint arXiv:1707.05005*.

Newman, Mark EJ, and Duncan J Watts. 1999. "Renormalization group analysis of the small-world network model." *Physics Letters A*, 263(4-6): 341–346.

Papamakarios, George, and Iain Murray. 2016. "Fast ε-free inference of simulation models with bayesian conditional density estimation." *Advances in neural information processing systems*, 29.

Robins, Garry, Pip Pattison, Yuval Kalish, and Dean Lusher. 2007. "An introduction to exponential random graph (p*) models for social networks." *Social networks*, 29(2): 173–191.

Rozemberczki, Benedek, and Rik Sarkar. 2020. "Characteristic functions on graphs: Birds of a feather, from statistical descriptors to parametric models." 1325–1334.

Rozemberczki, Benedek, Oliver Kiss, and Rik Sarkar. 2020. "Karate Club: An API Oriented Open-source Python Framework for Unsupervised Learning on Graphs." 3125–3132, ACM.

Veličković, Petar, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. "Graph attention networks." *arXiv preprint arXiv:1710.10903*.

Wang, Lili, Chenghan Huang, Weicheng Ma, Xinyuan Cao, and Soroush Vosoughi. 2021. "Graph Embedding via Diffusion-Wavelets-Based Node Feature Distribution Characterization." 3478–3482.

Wu, Z, S Pan, F Chen, G Long, C Zhang, and PS Yu. 2020. "A Comprehensive Survey on Graph Neural Networks." *IEEE Transactions on Neural Networks and Learning Systems.*

Xu, Keyulu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. "How powerful are graph neural networks?" *arXiv preprint arXiv:1810.00826.*

Department of Economics, University of Michigan, Ann Arbor